

IOT ELECTIVE PITCH SESSION

Medical Clinic Digital Twin

BLE-Based Real-Time Patient Tracking & Workflow Analytics

Khalid Ashmawy / Ahmed Khedr / Abdalla Mohamed / Ahmed Farag / Abdullah Sherif

April 30, 2026 | Supervisor: Prof. Tallal El-Shabrawy

| AGENDA

01. Introduction & Problem Statement

02. Literature Review (Related Work)

03. Existing Solutions & Their Limitations

04. Proposed "Central Link" Solution

05. Architecture & Localization Engine

06. Digital Twin Dashboard Demo

07. Implementation Plan & Hardware Progress

08. Challenges, References & Q&A

THE PROBLEM: OPERATIONAL BLINDNESS

Fragmented Patient Tracking

Medical staff currently cannot monitor precise patient movement between reception, diagnostic labs, and consultation rooms.

- **Inefficient Flow:** Waiting rooms overflow while staff are underutilized.
- **High PTAT:** Long Patient Turnaround Time negatively impacts health outcomes.
- **Static Data:** Manual check-ins miss real-time bottlenecks.



LITERATURE REVIEW (RELATED WORK)



Optimization of PTAT via BLE-Based RTLS

Arumugam & Muthaiyah (2024) developed a proof-of-concept utilizing Bluetooth Low Energy (BLE) beacons to optimize Patient Turnaround Time in public hospitals.

- **Outcome:** Their system successfully reduced total waiting times by **33% to 35%**.
- **Method:** Real-time localization using BLE signals to identify when a patient enters specific hospital zones.

Citation: Arumugam, G. R., & Muthaiyah, S. (2024). A Proof of Concept Using BLE... IJISRT, 9(5).

EXISTING SOLUTIONS & SOURCED LIMITATIONS

Current Limitations

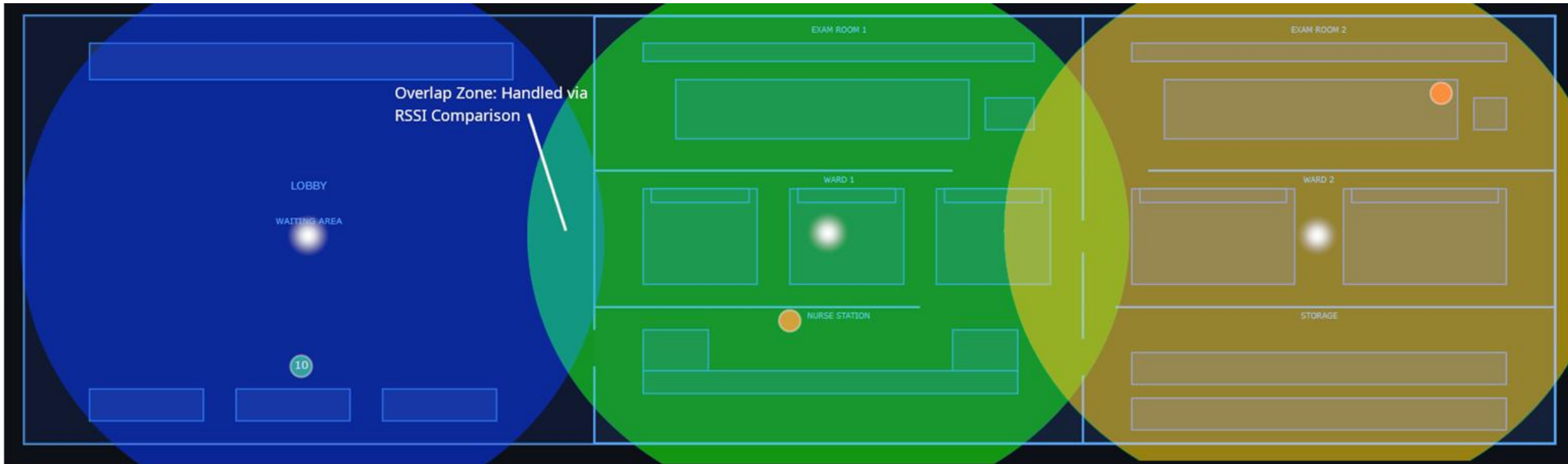
- **Manual Tracking:** High human error (22.5% inaccuracy) and paperwork overhead. (*Abdulmalek et al., 2022*)
- **RFID (Passive):** Prohibitive deployment costs and extremely short detection range. (*Yao et al., 2010*)
- **GPS/UWB:** Ineffective indoors due to signal attenuation through ceilings and walls. (*Ullah et al., 2025*)
- **WiFi:** High computational and energy costs for battery-constrained wearables. (*Hailu et al., 2025*)

Our "Central Link" Solution

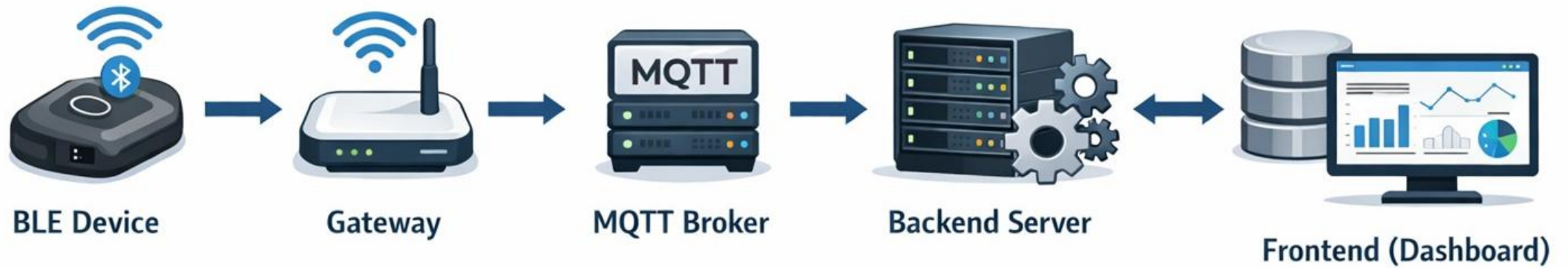
- **Live Digital Twin:** Resolves "operational blindness" via a 2D blueprint map.
- **Secure MQTTS:** Mandatory TLS 1.2/1.3 encryption for HIPAA-grade privacy.
- **AI Anomaly Detection:** Proactive identification of loitering using Isolation Forest models.
- **Scalability:** Low-power consumption suitable for lightweight clinical wearable tags.

THE DIGITAL TWIN MAPPING ENGINE

- **2D Blueprint Translation:** Translating raw gateway data into a live architectural floor plan.
- **Room-Level Accuracy:** Pinpointing patients to specific wards, exam rooms, or the lobby based on nearest-node proximity.



HIGH-LEVEL SYSTEM ARCHITECTURE



BLE Device → Gateway → Gateway → MQTT Broker → MQTT Broker → Backend Server

PROPOSED SOLUTION: CENTRAL LINK



Precision Edge

ESP32 Gateways scanning BLE advertising packets, ensuring low power consumption for wearable patient tags.



Secure MQTTS

Telemetry data is encrypted using dedicated SSL certificates before reaching our Ubuntu-based cloud VM.

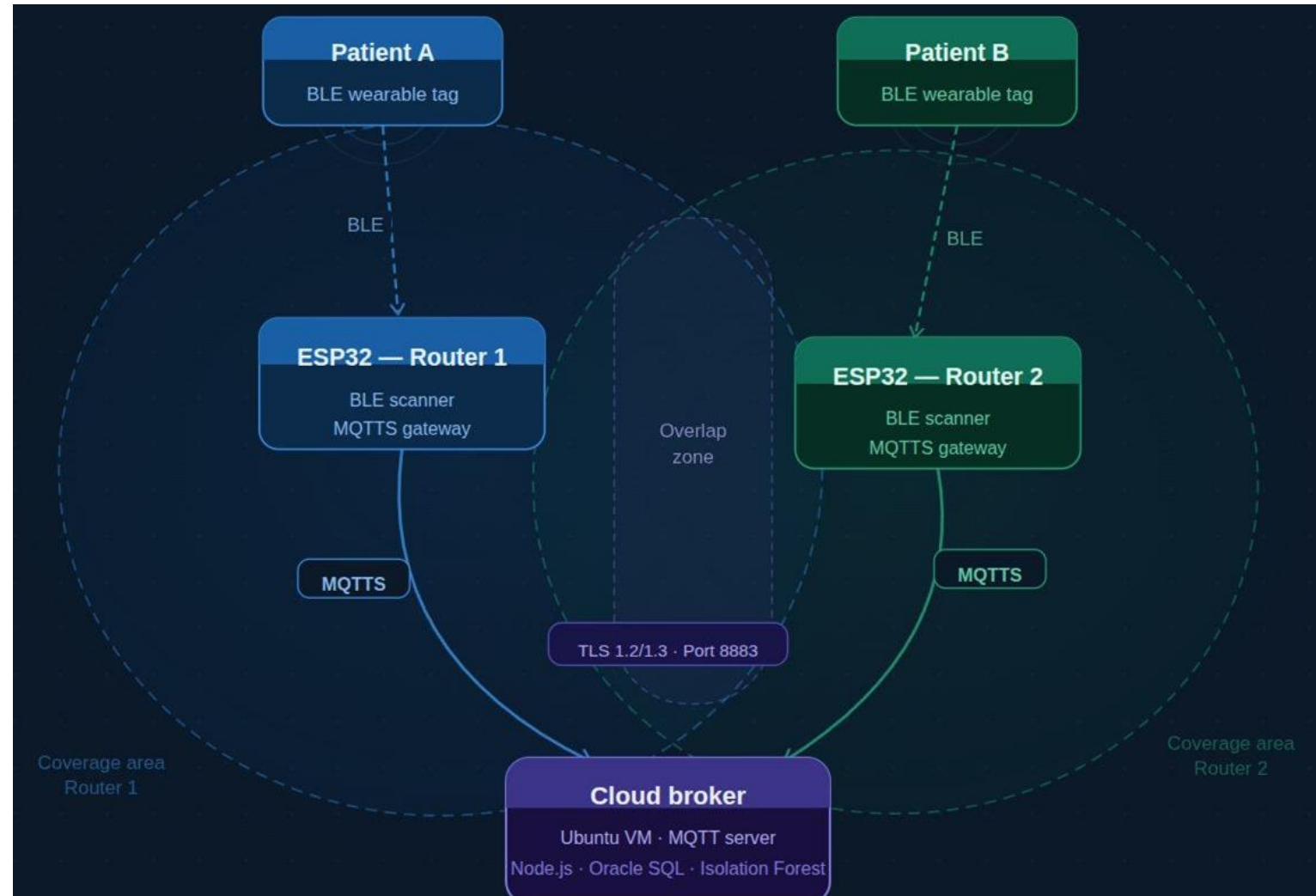


Predictive Analytics

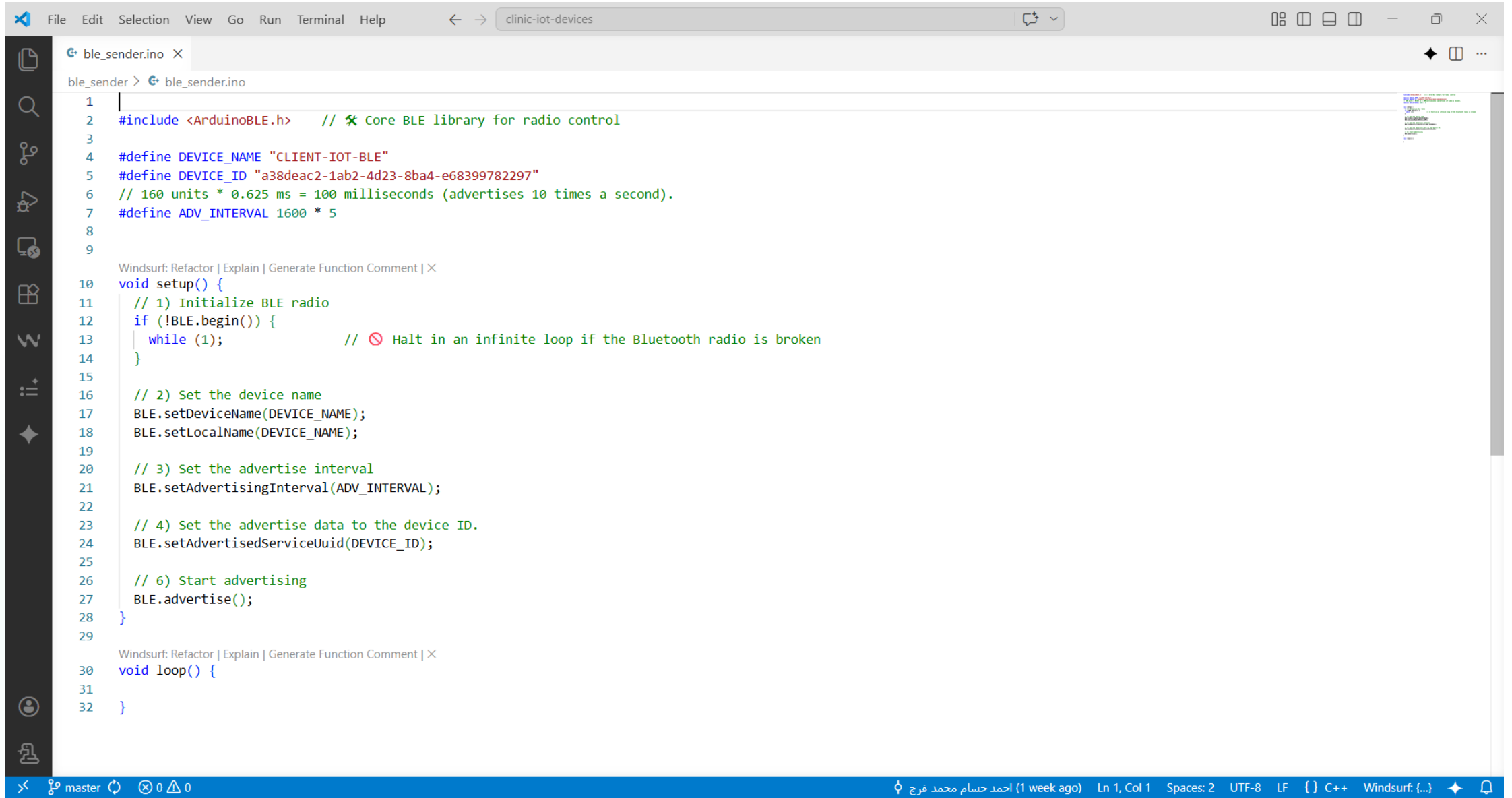
Node.js backend processes data for a Digital Twin dashboard with live heatmaps and bottleneck.

ARCHITECTURE DEEP DIVE & ZONE RESOLUTION

- **Secure Telemetry:** Data routed through Port 8883 using mandatory TLS 1.2/1.3 encryption.
- **Zone Resolution:** Handling overlapping gateway coverage areas by comparing real-time RSSI signal strength.
- **Cloud Processing:** Ubuntu VM hosting the Node.js backend, Oracle SQL database, and ML anomaly models.



Devices Setup

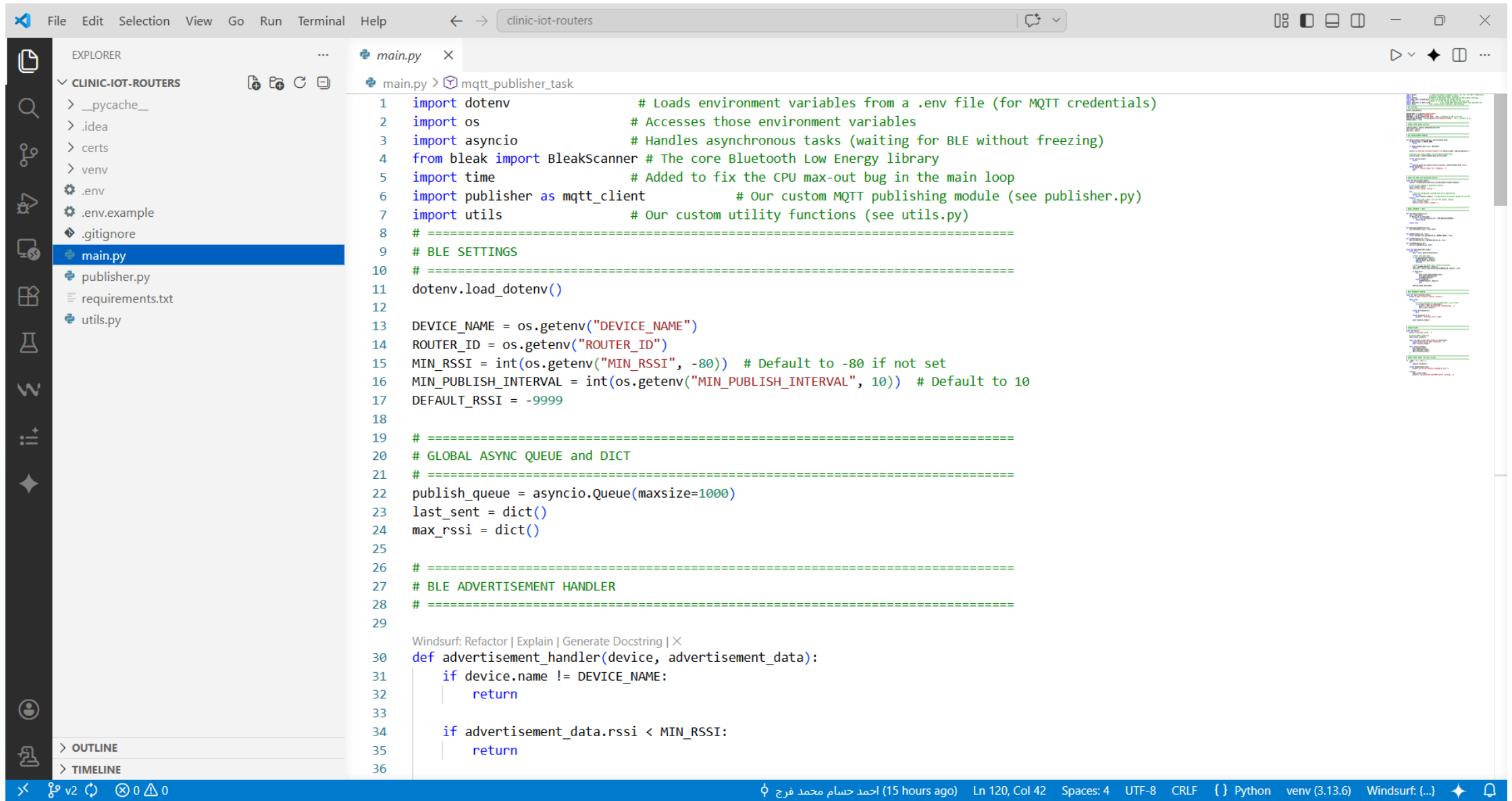


The screenshot shows a code editor window with the title bar "clinic-iot-devices". The editor is open to a file named "ble_sender.ino". The code is written in C++ and defines a BLE device named "CLIENT-IOT-BLE" with a specific ID and advertising interval. The setup function initializes the BLE radio, sets the device name and local name, sets the advertising interval, sets the advertising data to the device ID, and starts advertising. The loop function is currently empty.

```
1 |
2 | #include <ArduinoBLE.h>    // ✖ Core BLE library for radio control
3 |
4 | #define DEVICE_NAME "CLIENT-IOT-BLE"
5 | #define DEVICE_ID "a38deac2-1ab2-4d23-8ba4-e68399782297"
6 | // 160 units * 0.625 ms = 100 milliseconds (advertises 10 times a second).
7 | #define ADV_INTERVAL 1600 * 5
8 |
9 |
10 | Windsurf: Refactor | Explain | Generate Function Comment | ✕
11 | void setup() {
12 |     // 1) Initialize BLE radio
13 |     if (!BLE.begin()) {
14 |         while (1);           // ⚠ Halt in an infinite loop if the Bluetooth radio is broken
15 |     }
16 |
17 |     // 2) Set the device name
18 |     BLE.setDeviceName(DEVICE_NAME);
19 |     BLE.setLocalName(DEVICE_NAME);
20 |
21 |     // 3) Set the advertise interval
22 |     BLE.setAdvertisingInterval(ADV_INTERVAL);
23 |
24 |     // 4) Set the advertise data to the device ID.
25 |     BLE.setAdvertisedServiceUuid(DEVICE_ID);
26 |
27 |     // 6) Start advertising
28 |     BLE.advertise();
29 | }
30 |
31 | Windsurf: Refactor | Explain | Generate Function Comment | ✕
32 | void loop() {
33 | }
34 |
```

The status bar at the bottom shows the current branch is "master", there are 0 commits, 0 issues, and 0 pull requests. The editor is using the C++ language and the Windsurf editor.

Routers Setup



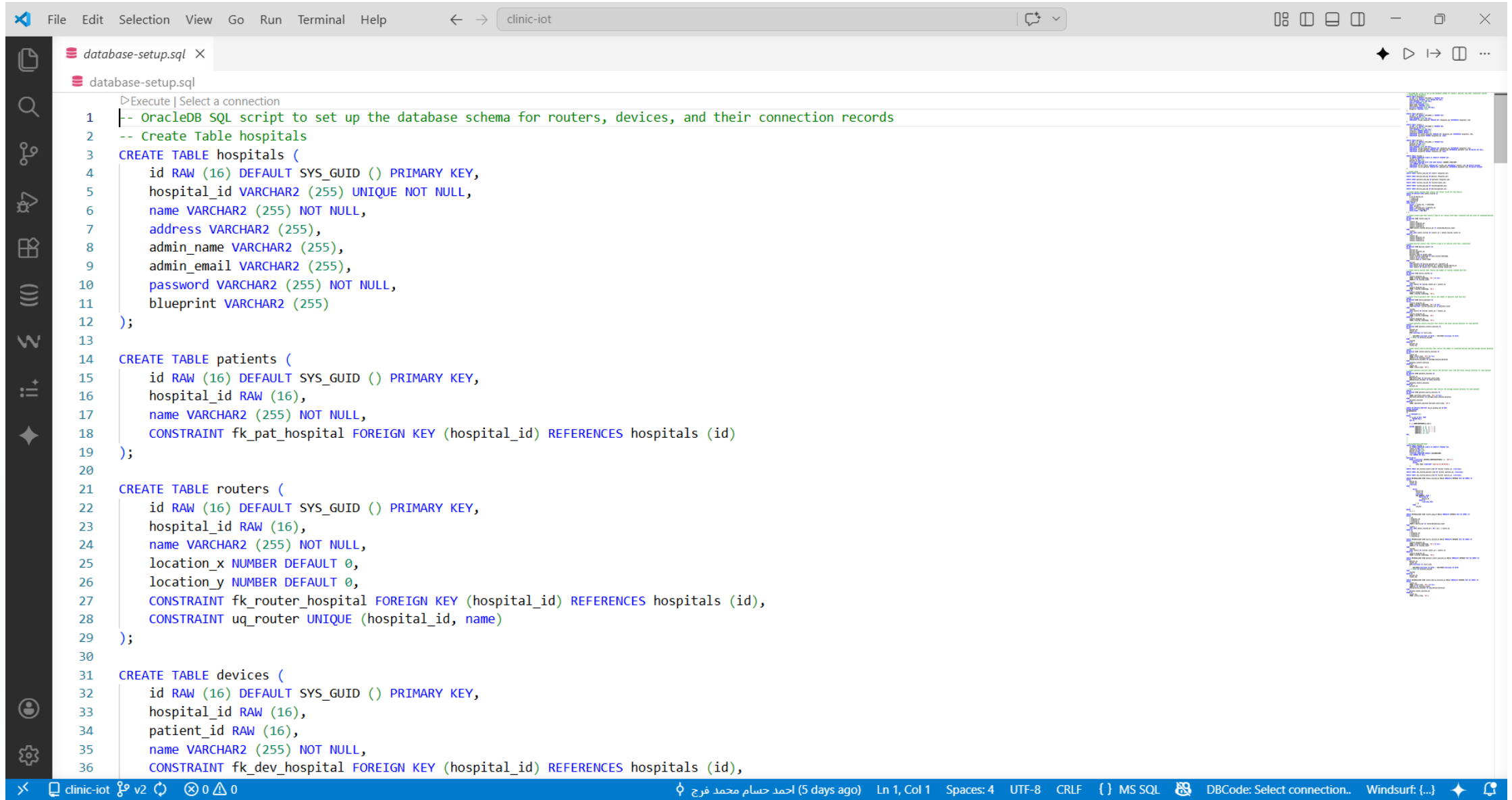
```
1 import dotenv # Loads environment variables from a .env file (for MQTT credentials)
2 import os # Accesses those environment variables
3 import asyncio # Handles asynchronous tasks (waiting for BLE without freezing)
4 from bleak import BleakScanner # The core Bluetooth Low Energy library
5 import time # Added to fix the CPU max-out bug in the main loop
6 import publisher as mqtt_client # Our custom MQTT publishing module (see publisher.py)
7 import utils # Our custom utility functions (see utils.py)
8 # =====
9 # BLE SETTINGS
10 # =====
11 dotenv.load_dotenv()
12
13 DEVICE_NAME = os.getenv("DEVICE_NAME")
14 ROUTER_ID = os.getenv("ROUTER_ID")
15 MIN_RSSI = int(os.getenv("MIN_RSSI", -80)) # Default to -80 if not set
16 MIN_PUBLISH_INTERVAL = int(os.getenv("MIN_PUBLISH_INTERVAL", 10)) # Default to 10
17 DEFAULT_RSSI = -9999
18
19 # =====
20 # GLOBAL ASYNC QUEUE and DICT
21 # =====
22 publish_queue = asyncio.Queue(maxsize=1000)
23 last_sent = dict()
24 max_rssi = dict()
25
26 # =====
27 # BLE ADVERTISEMENT HANDLER
28 # =====
29
30 def advertisement_handler(device, advertisement_data):
31     if device.name != DEVICE_NAME:
32         return
33
34     if advertisement_data.rssi < MIN_RSSI:
35         return
36
```

| MQTT Broker

- **Eclipse Mosquitto MQTT Server:** an open source (EPL/EDL licensed) message broker that implements the MQTT protocol versions 5.0, 3.1.1 and 3.1. Mosquitto is lightweight and is suitable for use on all devices from low power single board computers to full servers.



Database Setup



The screenshot shows a code editor window titled 'clinic-iot' with a file named 'database-setup.sql'. The script is an OracleDB SQL script designed to set up a database schema for routers, devices, and their connection records. It includes comments and SQL commands to create three tables: 'hospitals', 'patients', and 'routers', each with specific columns and constraints.

```
1  |-- OracleDB SQL script to set up the database schema for routers, devices, and their connection records
2  -- Create Table hospitals
3  CREATE TABLE hospitals (
4      id RAW (16) DEFAULT SYS_GUID () PRIMARY KEY,
5      hospital_id VARCHAR2 (255) UNIQUE NOT NULL,
6      name VARCHAR2 (255) NOT NULL,
7      address VARCHAR2 (255),
8      admin_name VARCHAR2 (255),
9      admin_email VARCHAR2 (255),
10     password VARCHAR2 (255) NOT NULL,
11     blueprint VARCHAR2 (255)
12 );
13
14 CREATE TABLE patients (
15     id RAW (16) DEFAULT SYS_GUID () PRIMARY KEY,
16     hospital_id RAW (16),
17     name VARCHAR2 (255) NOT NULL,
18     CONSTRAINT fk_pat_hospital FOREIGN KEY (hospital_id) REFERENCES hospitals (id)
19 );
20
21 CREATE TABLE routers (
22     id RAW (16) DEFAULT SYS_GUID () PRIMARY KEY,
23     hospital_id RAW (16),
24     name VARCHAR2 (255) NOT NULL,
25     location_x NUMBER DEFAULT 0,
26     location_y NUMBER DEFAULT 0,
27     CONSTRAINT fk_router_hospital FOREIGN KEY (hospital_id) REFERENCES hospitals (id),
28     CONSTRAINT uq_router UNIQUE (hospital_id, name)
29 );
30
31 CREATE TABLE devices (
32     id RAW (16) DEFAULT SYS_GUID () PRIMARY KEY,
33     hospital_id RAW (16),
34     patient_id RAW (16),
35     name VARCHAR2 (255) NOT NULL,
36     CONSTRAINT fk_dev_hospital FOREIGN KEY (hospital_id) REFERENCES hospitals (id),
```

Backend

File Edit Selection View Go Run Terminal Help

EXPLORER

CLINIC-IOT-BACKEND

- controllers
- converters
 - converters.js
- queries
 - auth.js
 - devices.js
 - patients.js
 - records.js
 - routers.js
- validators
 - auth.js
 - devices.js
 - patients.js
 - routers.js
 - settings.js
- middleware
 - auth.js
- node_modules
- routes
 - auth.js
 - devices.js
 - patients.js
 - records.js
 - routers.js

endpoints.json

```
1 {
2   "version": "1.0",
3   "description": "API endpoints for managing and retrieving information about routers, devices, and records.",
4   "note": "All durations are in seconds.",
5   "keys description": {
6     "path": "The URL path for the API endpoint.",
7     "description": "A brief description of what the endpoint does.",
8     "response": "The expected response format for the endpoint."
9   },
10  "endpoints": [
11    {
12      "path": "/auth/signup",
13      "method": "POST",
14      "description": "Sign up a new hospital.",
15      "body": {
16        "hospital_id": "VarChar(255)",
17        "name": "VarChar(255)",
18        "address": "VarChar(255)",
19        "admin_name": "VarChar(255)",
20        "admin_email": "VarChar(255)",
21        "password": "VarChar"
22      }
23    },
24    {
25      "path": "/auth/login",
26      "method": "POST",
27      "description": "Log in to an existing hospital.",
28      "body": {
29        "hospital_id": "VarChar(255)",
30        "password": "VarChar"
31      }
32    },
33    {
34      "path": "/settings",
35      "method": "GET",
36      "description": "Get the hospital settings.",
37      "response": {
```

Ln 71, Col 22 Spaces: 4 UTF-8 CRLF {} JSON Windsurf: {} Gemini Code Assist: Indexing workspace...

Dashboard

The image shows a Visual Studio Code editor window with a project named "CLINIC-IOT-FRONTEND". The Explorer sidebar on the left lists the project structure, including files like `__pycache__`, `.venv`, `assets`, `pages`, `analytics.py`, `dashboard.py`, `device_management.py`, `login.py`, `patient_stats.py`, `router_management.py`, `router_stats.py`, `settings.py`, `user_tracking.py`, `.env`, `.env.example`, `.gitignore`, `api.py`, `app.py` (selected), `floor_plan.py`, `gunicorn.conf.py`, `README.md`, and `requirements.txt`. The main editor area displays the content of `app.py`, which is a Python script using the Flask-Dash framework. The code includes imports for `dash`, `flask`, `api`, `dotenv`, and `os`. It sets up a Flask application with a secret key and a Dash application with various settings like `use_pages=True` and `suppress_callback_exceptions=True`. The `NAV_ITEMS` list defines the navigation menu with icons and links to different pages. A status bar at the bottom indicates the current file is `app.py` on line 13, column 48, with 4 spaces, UTF-8 encoding, and CRLF line endings.

```
1 import dash
2 from dash import dcc, html, Input, Output, State
3 import flask
4 import api
5 import dotenv
6 import os
7
8
9 dotenv.load_dotenv()
10
11 SERVER_SECRET = (os.getenv("SERVER_SECRET") or "").strip()
12 SERVER_HOST = (os.getenv("SERVER_HOST", "127.0.0.1") or "").strip()
13 SERVER_PORT = int(os.getenv("SERVER_PORT", 8050))
14 DEBUG_MODE = os.getenv("DEBUG_MODE", "false").strip().lower() == "true"
15
16
17 server = flask.Flask(__name__)
18 server.secret_key = SERVER_SECRET
19
20 app = dash.Dash(
21     __name__,
22     server=server,
23     use_pages=True,
24     suppress_callback_exceptions=True,
25     external_stylesheets=[],
26     title="Central Link - IoT Management",
27 )
28
29 NAV_ITEMS = [
30     ("🏠", "Dashboard", "/"),
31     ("🔧", "Router Management", "/routers"),
32     ("📡", "Device Management", "/devices"),
33     ("👤", "Patient Tracking", "/patients"),
34     ("📊", "Analytics", "/analytics"),
35     ("⚙️", "Settings", "/settings"),
36 ]
37
```

SECURE AUTHENTICATION



Central Link

Next-Gen IoT & AI Monitoring Dashboard

Seamlessly integrate medical devices, monitor patient vitals in real-time, and leverage predictive AI analytics for your healthcare facility.

- HIPAA Compliant Infrastructure
- Millisecond Data Latency
- AI-Driven Predictive Alerts

Welcome Back

Access your secure clinical workspace

HOSPITAL ID

HOSPITAL-SEED

PASSWORD

Initialize Secure Session →

Facility Onboarding

Register your hospital for the AI monitoring suite

HOSPITAL NAME

St. Margaret's Medical C

HOSPITAL ID

HOSP-0001

ADDRESS

123 Healthcare Ave, New Cairo

ADMIN NAME

Dr. Sarah Chen

ADMIN EMAIL

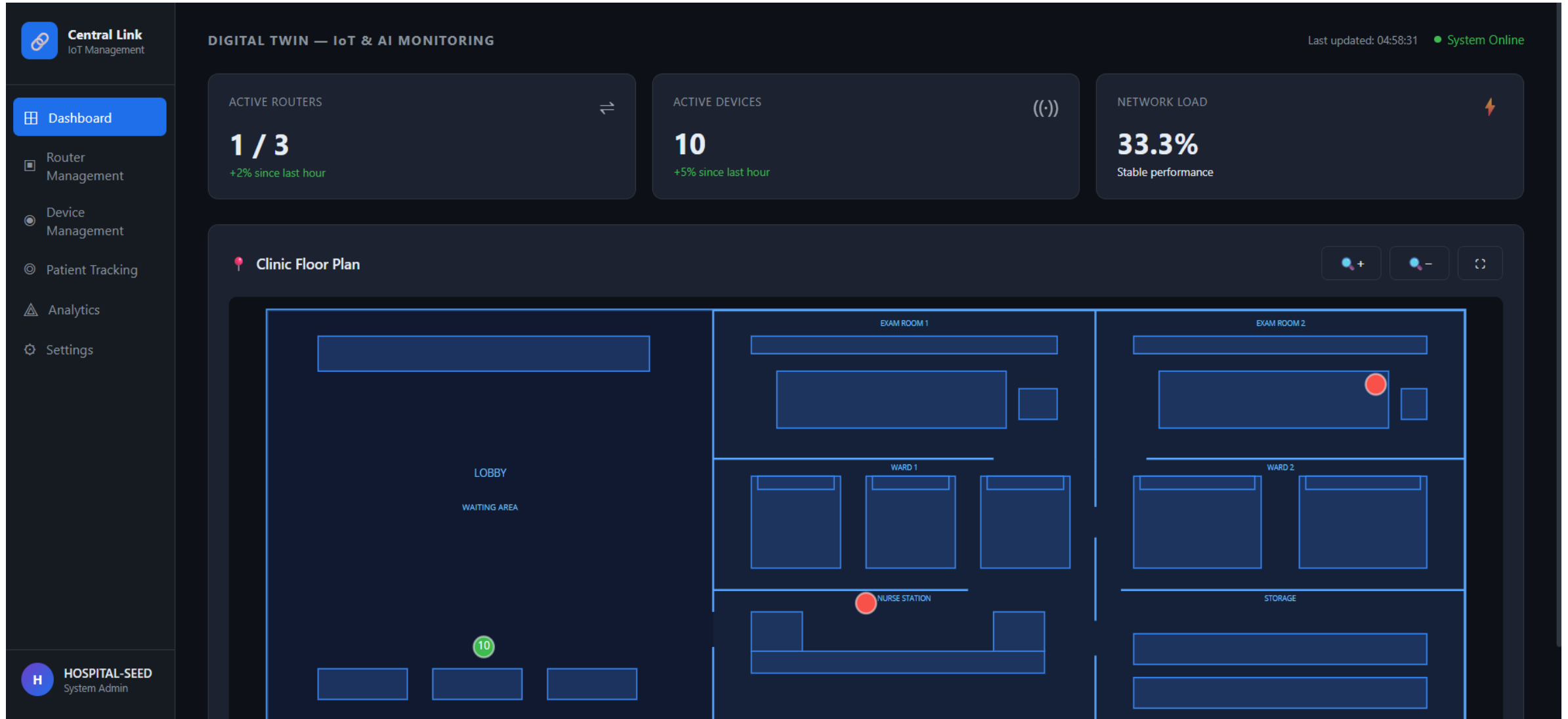
s.chen@hospital.com

ADMIN PASSWORD

Create a strong password

Request Facility Access

THE LIVE DASHBOARD



INFRASTRUCTURE (ROUTER MANAGEMENT)

Central Link
IoT Management

Dashboard

Router Management

Device Management

Patient Tracking

Analytics

Settings

H HOSPITAL-SEED
System Admin

Search routers...

+ Add Router

TOTAL ONLINE
1

Router Management

Monitor and control your connected IoT infrastructure

All RoutersActiveOffline

Seed Router 1

5a5b140f-9f5a-42

SIGNAL
Strong

DEVICES
0

View Details

Stats

OFFLINE

Seed Router 1

DEVICES
0

STATUS
OFFLINE

SIGNAL
Strong

Hourly Sessions

Time	Sessions
07:00	1
19:00	1
22:00	3
10:00	1
05:00	1
03:00	1
13:00	1
08:00	1

Connected Devices (0)

DEVICE	HOLDER	LAST SEEN
No devices connected.		

Add Router

Click the map to set the router location

ROUTER ID
00000000-0000-0000-0000-000000000000

ROUTER NAME
Router Name

CLICK MAP TO SET LOCATION
Click anywhere to place router

LOBBY
WAITING AREA

EXAM ROOM 1

EXAM ROOM 2

WARD 1

WARD 2

NURSE STATION

STORAGE

No location selected — click the map

Add Router

HARDWARE PROVISIONING

Central Link
IoT Management

Dashboard

Router Management

Device Management

Patient Tracking

Analytics

Settings

H

HOSPITAL-SEED
System Admin

Search devices...

+ Add Device

Device Management

Manage BLE devices in your clinic network.

All Devices

Free

Assigned

ASSIGNED

Seed Device 1

e2cc9d10-3098-4684-9

STATUS

PATIENT

In Use

19188c57

Release

ASSIGNED

Seed Device 2

702cc6af-029

STATUS

PATIENT

In Use

31

Release

ASSIGNED

Seed Device 3

32be5d2b-835

STATUS

PATIENT

In Use

f633633a

Release

ASSIGNED

Seed Device 4

266b2822-70d5-4d83-a

STATUS

PATIENT

In Use

a8424516

Release

ASSIGNED

Seed Device 5

c182eb67-d51d-45b1-9

STATUS

PATIENT

In Use

547b9ba2

Release

ASSIGNED

Seed Device 6

38321c32-fcc2-468a-b

STATUS

PATIENT

In Use

ad3f02ab

Release

ASSIGNED

Seed Device 7

ec19bf53

STATUS

PATIENT

In Use

ec19bf53

Release

ASSIGNED

Seed Device 8

05feea4e

STATUS

PATIENT

In Use

05feea4e

Release

ASSIGNED

Seed Device 9

15dbe57b-77a0-4a8e-9

STATUS

PATIENT

In Use

05feea4e

Release

ASSIGNED

Seed Device 10

dc4becf1-908a-4f3e-b

STATUS

PATIENT

In Use

e943c14c

Release

FREE

0

ASSIGNED

10

Add Device

Register a new BLE device

DEVICE ID

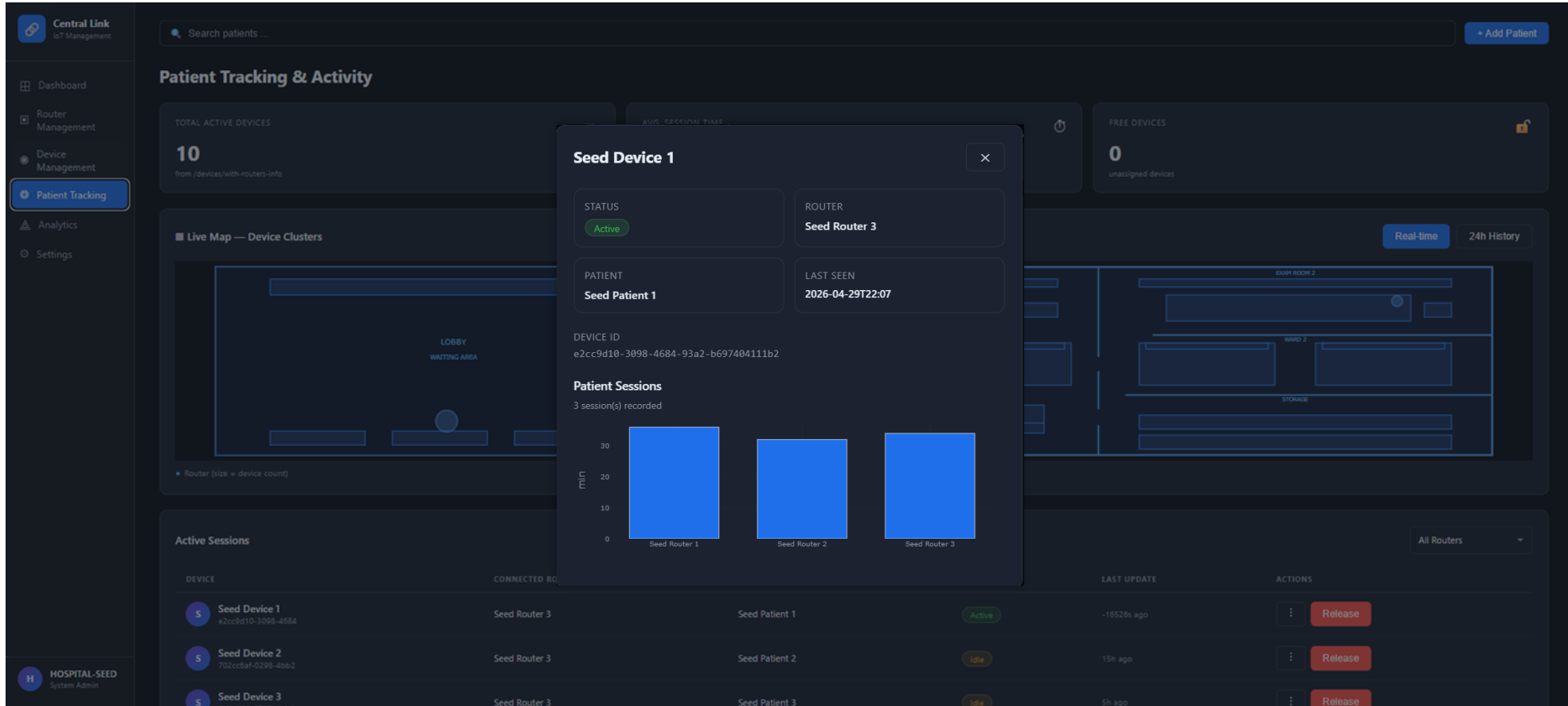
00000000-0000-0000-0000-00000000

DEVICE NAME

Device Name

Add Device

REAL-TIME PATIENT TRACKING



Add Patient

Assign a BLE device to a new patient

PATIENT NAME

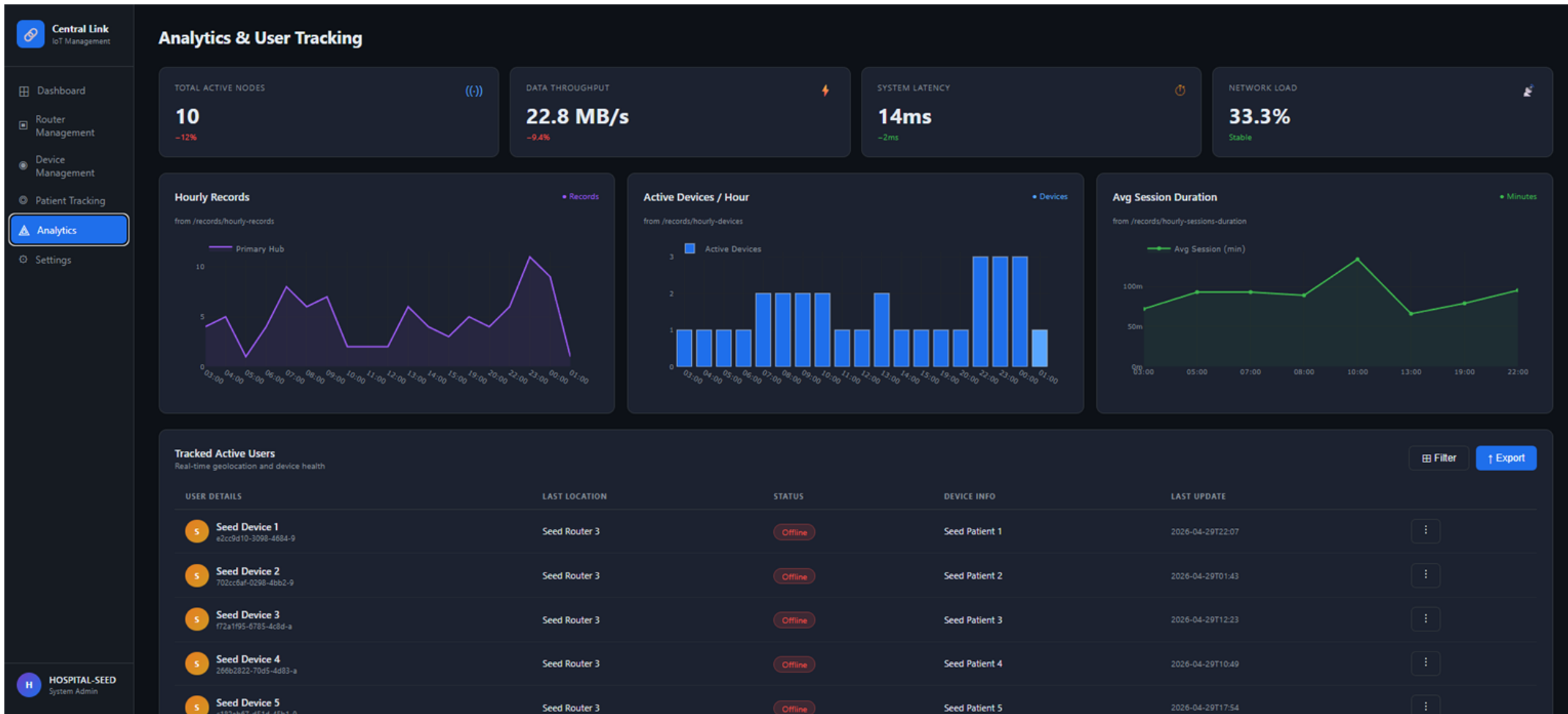
Patient Name

ASSIGN DEVICE

Select a free device...

[Add Patient](#)

WORKFLOW ANALYTICS



SYSTEM SETTINGS

Central Link
IoT Management

Dashboard

Router Management

Device Management

Patient Tracking

Analytics

Settings

System Settings

EditSave Changes

Hospital Profile

Click Edit to modify your facility details.

HOSPITAL NAME

Seed Hospital

FACILITY ID

HOSPITAL-SEED

PRIMARY ADDRESS

Seed Address

Admin Information

Click Edit to update contact information.

FULL NAME

Seed Admin

EMAIL ADDRESS

Gx212@example.com

Clinic Blueprint

Upload or update the floor plan used for router positioning.

No blueprint uploaded yet

Upload Blueprint

View Full Map

Password Reset

Ensure your account security by updating your password periodically.

CURRENT PASSWORD

NEW PASSWORD

CONFIRM NEW PASSWORD

Update Password

Data Management

Manage hospital patient records and tracking data.

Reset Patient Records

Removes all patient sessions and records. Routers and devices are kept.

Reset Records

Account Management

Irreversible actions regarding your system access.

Delete Administrator Account

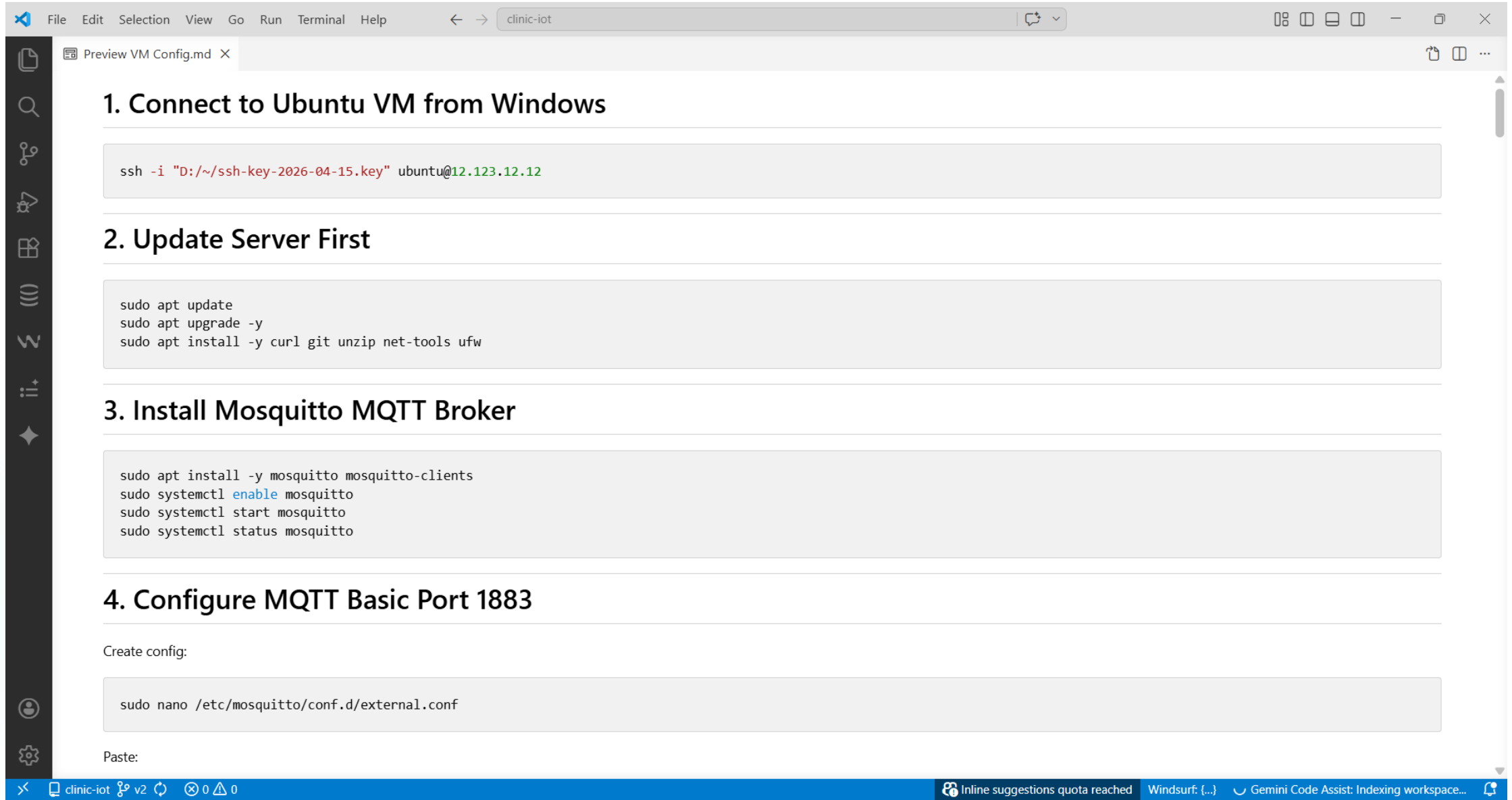
Once deleted, you will lose all access to the management console.

Delete Account

H HOSPITAL-SEED

System Admin

VM Setup



The screenshot shows a VS Code editor window with a file named "Preview VM Config.md". The editor displays a four-step guide for setting up an Ubuntu VM. The steps are: 1. Connect to Ubuntu VM from Windows, 2. Update Server First, 3. Install Mosquitto MQTT Broker, and 4. Configure MQTT Basic Port 1883. Each step includes a code block with the necessary commands. The interface includes a sidebar with icons for Explorer, Search, Source Control, Run and Debug, Extensions, Testing, Accounts, and Settings. The bottom status bar shows the current file is "clinic-iot v2" and includes a message about the inline suggestions quota.

```
File Edit Selection View Go Run Terminal Help clinic-iot
```

Preview VM Config.md X

1. Connect to Ubuntu VM from Windows

```
ssh -i "D:/~/ssh-key-2026-04-15.key" ubuntu@12.123.12.12
```

2. Update Server First

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y curl git unzip net-tools ufw
```

3. Install Mosquitto MQTT Broker

```
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
sudo systemctl status mosquitto
```

4. Configure MQTT Basic Port 1883

Create config:

```
sudo nano /etc/mosquitto/conf.d/external.conf
```

Paste:

clinic-iot v2 0 0 0

Inline suggestions quota reached Windsurf: {...} Gemini Code Assist: Indexing workspace...

| Domain

<https://central-link-iot.duckdns.org>

CHALLENGES & FUTURE WORK

Technical Hurdles

Signal Multipathing: Indoor noise creates RSSI fluctuations.

Latency: Maintaining sub-100ms updates.

Final Deliverables

ML Anomaly Module: Training anomaly detection models to flag unexpected patient loops.

Scale Testing: Validating system performance with 20+ active tags.

ACADEMIC REFERENCES

- Arumugam, G. R., & Muthaiyah, S. (2024). A Proof of Concept Using BLE to Optimize Patients' Turnaround Time... [IJISRT.](#)
- Abdulmalek, S., Nasir, A., Jabbar, W. A., et al. (2022). IoT-Based Healthcare-Monitoring System towards Improving Quality of Life: A Review. Healthcare (Basel).
- Ullah, S., et al. (2025). Advancing indoor positioning systems: innovations, challenges... [Robotica.](#)
- Hailu, T. G., Guo, X., & Si, H. (2025). Indoor Positioning Systems as Critical Infrastructure... [Sensors.](#)
- Yao, W., Chu, C. H., & Li, Z. (2010). The use of RFID in healthcare: Benefits and barriers. [Research Database.](#)
- Grieves, M., & Vickers, J. (2017). Digital Twin: Mitigating Unpredictable... [Springer, Cham.](#)

Thank You!

Questions & Discussion

Central Link Team | Medical Clinic Digital Twin

April 30 Pitch